Programming Language Principles, Design, and Implementation

Compilers - Intermediate Representations

Vincent Rahli

University of Birmingham

Where are we?

- Part 1: Principles of programming
- **Part 2: Compilers**

Today

Intermediate Representations

- Intermediate Representation generation
- TACs
- Control Flow Operators
- SSA

Further reading:

- Chapter 7 of "Modern Compiler Implementation"
- Section 6 of the Dragon book

Problem

The lexing, syntactic, and semantic analysis phases check for errors in the source program, and generate an AST that can be easily analyzed and transformed

Intermediate Representation (IR) generation

- final phase of the compiler front-end
- translates the AST into code expected by the back-end
- should allow compiling multiple source languages, to multiple target languages.
- multiple IRs can be used from high-level (e.g., syntax trees) to low level (e.g., three-address codes)

Three-Address Code

Three-Address Code (TAC) is a commonly used intermediate language

- instructions can refer to at most 3 addresses
- they contain at most one operation to the right of an instruction

For example: an expression of the form $a + b * c$ would be rewritten as

$$t_1 = b * c$$
$$t_2 = a + t_1$$

where t_1 and t_2 are temporary variables generated by the compiler

Three-Address Code

A TAC is a list of **instructions** referring to **addresses**

Notation: We write $x, y, z, \ldots$ for addresses, op for binary or unary arithmetic or logical operations **(must be rich enough to implement source code operations, easy enough to translate to target code)**; $relop$ for relational operators of the for $<$, $==$, $\geqslant$, etc.; L for labels; n for numbers

An address can be:

- a source program name (a pointer to its entry in the symbol table)
- a constant, e.g., a number
- a temporary variable generated by the compiler

An instruction can be:

- assignment: $x = y \ op \ z$ or $x = op \ y$
- copy: $x = y$ – **assigns the value of y to x**
- unconditional jump: `goto` L – **executes the instruction with label L next**
- ...

Three-Address Code

instructions (continued):

- ...
- conditional jump (the next instruction in the list is executed if the condition fails):
  - `if` x `goto` L – **executes the instruction with label** L **next if** x **is true**
  - `ifFalse` x `goto` L – **executes the instruction with label** L **next if** x **is false**
  - `if` x $relop$ y `goto` L – **executes the instruction with label** L **next if** $(x, y) \in relop$
  - `ifFalse` x $relop$ y `goto` L – **executes the instruction with label** L **next if** $(x, y) \notin relop$
- indexed copy:
  - $x = y[i]$ – **sets** x **to** y **offsetted by** i **memory units**
  - $x[i] = y$ – **sets** x **offsetted by** i **memory units to** y
- address and pointer assignments:
  - $x = \&y$ – **sets the value of** x **to be** y**'s location**
  - $x = *y$ – **sets the value of** x **to be the value at** y**'s location**
  - $*x = y$ – **sets the value at** x**'s location to be** y**'s value**
- ...

Three-Address Code

instructions (continued):

- ...
- procedure calls and returns:
  - `param` x – **parameter**
  - `call` p, n – **procedure call**
  - $y =$ `call` y, p – **function call**
  - `return` y – **where the return value y is optional**
- ...

Calls are typically used as follows:

```
param x_1
...
param x_n
call p, n
```

Defining a procedure:

- p labels its first statement
- its last statement is `return` y

Three-Address Code

Example: the following statement

$$\texttt{do } i = i + 1; \texttt{ while } (a[i] < v);$$

can be translated into (for an array a of elements taking 8 units of space):

$$\begin{array}{ll} L: & t_1 = i + 1 \\ & i = t_1 \\ & t_2 = i * 8 \\ & t_3 = a[t_2] \\ & \texttt{if } t_3 < v \texttt{ goto } L \end{array}$$

Translate `while` $(i < 10)\ \{a[i] = 0; i{+}{+};\}$ to TAC

$$\begin{array}{ll} L_1: & \texttt{ifFalse } i < 10 \texttt{ goto } L_2 \\ & t_1 = i * 8 \\ & a[t_1] = 0 \\ & i = i + 1 \\ & \texttt{goto } L_1 \\ L_2: & \ldots \end{array}$$

Three-Address Code

Instructions can be represented as: **quadruples**, **triples**, or **indirect triples**.

A **quadruple** has 4 fields: op (an internal code for the operator), arg_1, arg_2, $result$

- unary operations do not use arg_2
- `param` does not use arg_2 or $result$
- jumps put L in $result$

Example: consider the statement $a = b * -c + b * -c;$

TAC:

$$
\begin{aligned}
t_1 &= \texttt{minus}\ c \\
t_2 &= b * t_1 \\
t_3 &= \texttt{minus}\ c \\
t_4 &= b * t_3 \\
t_5 &= t_2 + t_4 \\
a &= t_5
\end{aligned}
$$

Quadruples:

	op	arg_1	arg_2	$result$
0	`minus`	c		t_1
1	$*$	b	t_1	t_2
2	`minus`	c		t_3
3	$*$	b	t_3	t_4
4	$+$	t_2	t_4	t_5
5	$=$	t_5		a

Three-Address Code

A **triple** has 3 fields: op, arg_1, and arg_2

- no temporary names in the $result$ field
- instead, results are referred to by their positions in the table
- e.g., t_1 is replaced by (0)

Example: consider the statement $a = b * -c + b * -c;$

TAC:

$$
\begin{aligned}
t_1 &= \texttt{minus}\ c \\
t_2 &= b * t_1 \\
t_3 &= \texttt{minus}\ c \\
t_4 &= b * t_3 \\
t_5 &= t_2 + t_4 \\
a &= t_5
\end{aligned}
$$

Triples:

	op	arg_1	arg_2
0	`minus`	c	
1	$*$	b	(0)
2	`minus`	c	
3	$*$	b	(2)
4	$+$	(1)	(3)
5	$=$	a	(4)

- The 1st argument of a copy is stored in arg_1 and the 2nd in arg_2
- Ternary instructions such as $x[i] = y$ require two triples

Three-Address Code

Quadruples vs. Triples:

- with quadruples, moving an instruction that computes a temporary t does not require any change to the instructions that refer to t
- with triples, moving an instruction may require updating pointers to positions

This is solved using **indirect triples**, which are lists of pointers to triples, listing the order of the triples (stored in a separate triple table).

To reorder instructions, only the list of pointers needs to be reordered, without affecting the triples.

Static Single-Assignment

Static Single-Assignment (SSA) is an IR that facilitates some optimizations, such as:

- dead-code elimination
- simple constant propagation
- ...

In SSA, **no two assignments are for the same variable**

- Each new definition of a variable is modified to define a fresh variable
- Each variable use is modified to use the most recently defined one

Example

TAC:

$$
\begin{aligned}
p &= a + b \\
q &= p - c \\
p &= q * d \\
p &= e - p \\
q &= p + q
\end{aligned}
$$

SSA:

$$
\begin{aligned}
p_1 &= a + b \\
q_1 &= p_1 - c \\
p_2 &= q_1 * d \\
p_3 &= e - p_2 \\
q_2 &= p_3 + q_1
\end{aligned}
$$

Static Single-Assignment

Problem: What if a program uses a control flow operator?

For example:

$$x = 1; \texttt{if } (z < 0)\ x = -1;$$
$$y = x;$$

Let us try to turn it into an SSA form:

$$x_1 = 1; \texttt{if } (z < 0)\ x_2 = -1;$$
$$y_1 = ?$$

There is no "most recently defined" x that can be computed statically, as it depends on the control operator!

A solution to solve this problem is to use a function ϕ called a **notational fiction**:

$$x_1 = 1; \texttt{if } (z < 0)\ x_2 = -1;$$
$$y_1 = \phi(x_1, x_2)$$

Here $\phi(x_1, x_2)$ returns x_2 if the "then" branch was entered and x_1 otherwise, which can be implemented using jumps.

Translation to Three-Address Code

AST:

$$\begin{array}{lcl} S & ::= & S; S \mid [S] \mid id \coloneqq E \\ E & ::= & E + E \mid E * E \mid num \mid id \end{array}$$

Statements and expressions can be translated to TAC as follows, where

- `tac` returns a TAC on statements
- it returns a pair of an address and a TAC on expressions
- $\|$ concatenates 2 TACs
- `lookup`(i) returns a pointers to i's entry in the symbol table
- t is a new temporary name

$$\begin{array}{lcl} \texttt{tac}(S_1; S_2) & = & \texttt{tac}(S_1) \parallel \texttt{tac}(S_2) \\ \texttt{tac}([S]) & = & \texttt{tac}(S) \\ \texttt{tac}(id \coloneqq E) & = & c \parallel x = a \\ \quad \text{where } \texttt{tac}(E) = \langle a, c \rangle & & \\ \quad \text{and } x = \texttt{lookup}(id) & & \end{array}$$

$$\begin{array}{lcl} \texttt{tac}(E_1 + E_2) & = & \langle t, c_1 \parallel c_2 \parallel t = a_1 + a_2 \rangle \\ \quad \text{where } \texttt{tac}(E_1) = \langle a_1, c_1 \rangle & & \\ \quad \text{and } \texttt{tac}(E_2) = \langle a_2, c_2 \rangle & & \\ \texttt{tac}(E_1 * E_2) & = & \langle t, c_1 \parallel c_2 \parallel t = a_1 * a_2 \rangle \\ \quad \text{where } \texttt{tac}(E_1) = \langle a_1, c_1 \rangle & & \\ \quad \text{and } \texttt{tac}(E_2) = \langle a_2, c_2 \rangle & & \\ \texttt{tac}(num) & = & \langle t, t = num \rangle \\ \texttt{tac}(id) & = & \langle \texttt{lookup}(id), \rangle \end{array}$$

Translation to Three-Address Code

Translate $x := 1; y := x$ to TAC

$$\begin{aligned}
\texttt{tac}(x := 1; y := x) &= \texttt{tac}(x := 1) \parallel \texttt{tac}(y := x) \\
&= c_1 \parallel i = a_1 \parallel c_2 \parallel j = a_2
\end{aligned}$$

where $\texttt{tac}(1) = \langle a_1, c_1 \rangle$
and $i = \texttt{lookup}(x)$
and $\texttt{tac}(x) = \langle a_2, c_2 \rangle$
and $j = \texttt{lookup}(y)$

$$= t_1 = 1 \parallel i = t_1 \parallel j = i$$

Control Flow

As hinted at above, **control flow** operators are challenging. They often rely on **Booleans** (if-then-else; while; etc.), which are used to **alter the control flow**.

We consider the following simple syntax for **Boolean expressions**:

$$\begin{array}{rcl} B & ::= & \texttt{true} \mid \texttt{false} \mid E < E \mid E > E \mid E == E \mid E \mathrel{!}= E \\ & \mid & B \mid\mid B \mid B \mathbin{\&\&} B \mid \mathrel{!}B \end{array}$$

We also add control flow operators:

$$\begin{array}{rcl} P & ::= & S \\ S & ::= & \cdots \mid \texttt{if}\ (B)\ S \mid \texttt{if}\ (B)\ S\ \texttt{else}\ S \mid \texttt{while}\ (B)\ S \end{array}$$

- When translating a statement S, we also pass the label of the instruction immediately after S
- When translating a Boolean B, we also pass 2 labels:
  - one to which control flows if B is true
  - one to which control flows if B is false

Control Flow

Translation of statements:

$$
\begin{aligned}
&\texttt{tac}(P) && = && \texttt{tac}(S, L) \parallel L \\
&\quad \text{where } P \text{ is } S \text{ and } L \text{ is a new label} \\
&\texttt{tac}(S_1; S_2, L) && = && \texttt{tac}(S_1, L') \parallel L' \parallel \texttt{tac}(S_2, L) \\
&\quad \text{where } L' \text{ is a new label} \\
&\texttt{tac}([S], L) && = && \texttt{tac}(S, L) \\
&\texttt{tac}(id := E, L) && = && c \parallel \texttt{lookup}(id) = a \\
&\quad \text{where } \texttt{tac}(E) = \langle a, c \rangle \\
&\texttt{tac}(\texttt{if } (B)\ S, L) && = && c \parallel L' \parallel \texttt{tac}(S, L) \\
&\quad \text{where } L' \text{ is a new label and } \texttt{tac}(B, L', L) = c \\
&\texttt{tac}(\texttt{if } (B)\ S_1\ \texttt{else}\ S_2, L) && = && c \parallel L_1 \parallel \texttt{tac}(S_1, L) \parallel \texttt{goto}\ L \parallel L_2 \parallel \texttt{tac}(S_2, L) \\
&\quad \text{where } L_1, L_2 \text{ are new labels and } \texttt{tac}(B, L_1, L_2) = c \\
&\texttt{tac}(\texttt{while } (B)\ S, L) && = && L_1 \parallel c \parallel L_2 \parallel \texttt{tac}(S, L_1) \parallel \texttt{goto}\ L_1 \\
&\quad \text{where } L_1, L_2 \text{ are new labels and } \texttt{tac}(B, L_2, L) = c
\end{aligned}
$$

Why is the **goto** instruction in the **while** rule necessary?

Control Flow

Consider the following statement: $(\texttt{while}\ (B)\ id \coloneqq 3); S$

`tac` translates it to:

$$
\begin{array}{ll}
L_1: & b_1 \\
 & \ldots \\
 & b_n \\
L_2: & x = 3 \\
 & \texttt{goto}\ L_1 \\
L3: & s_1 \\
 & \ldots \\
 & s_m
\end{array}
$$

where

- $\texttt{lookup}(id) = x$
- $b_1 \ldots b_n$ is the code generated from B
- $s_1 \ldots s_m$ is the code generated from S

Without the **goto** instruction, S's code would be executed right after the assignment, as if S was inside while's body.

Control Flow

Translation of Booleans:

$$\begin{aligned}
&\texttt{tac}(\texttt{true}, L_1, L_2) && = && \texttt{goto } L_1 \\
&\texttt{tac}(\texttt{false}, L_1, L_2) && = && \texttt{goto } L_2 \\
&\texttt{tac}(E_1 < E_2, L_1, L_2) && = && c_1 \parallel c_2 \parallel \texttt{if } a_1 < a_2 \texttt{ goto } L_1 \parallel \texttt{goto } L_2 \\
&\quad \text{and } \texttt{tac}(E_1) = \langle a_1, c_1 \rangle \\
&\quad \text{and } \texttt{tac}(E_2) = \langle a_2, c_2 \rangle \\
&\texttt{tac}(B_1 \,||\, B_2, L_1, L_2) && = && c_1 \parallel L' \parallel c_2 \\
&\quad \text{where } L' \text{ is a new label} \\
&\quad \text{where } \texttt{tac}(B_1, L_1, L') = c_1 \\
&\quad \text{and } \texttt{tac}(B_2, L_1, L_2) = c_2 \\
&\texttt{tac}(B_1 \,\&\&\, B_2, L_1, L_2) && = && c_1 \parallel L' \parallel c_2 \\
&\quad \text{where } L' \text{ is a new label} \\
&\quad \text{where } \texttt{tac}(B_1, L', L_2) = c_1 \\
&\quad \text{and } \texttt{tac}(B_2, L_1, L_2) = c_2 \\
&\texttt{tac}(!B, L_1, L_2) && = && c \\
&\quad \text{where } \texttt{tac}(B, L_2, L_1) = c
\end{aligned}$$

Control Flow

What does `if` $(x < 0 \,||\, x > 200 \,\&\&\, x \mathrel{!}= y)\; x := 0$ translate into?

It translates into

$$
\begin{array}{ll}
 & \texttt{if } x < 0 \texttt{ goto } L_1 \\
 & \texttt{goto } L_1' \\
L_1': & \texttt{if } x > 200 \texttt{ goto } L_2' \\
 & \texttt{goto } L_2 \\
L_2': & \texttt{if } x \mathrel{!}= y \texttt{ goto } L_1 \\
 & \texttt{goto } L_2 \\
L_1: & x = 0 \\
L_2: &
\end{array}
$$

Note that this is **not optimal**. For example:

- $\begin{array}{ll} & \texttt{goto } L \\ L: & \ldots \end{array}$ can be converted into $L: \ldots$
- and $\begin{array}{ll} & \texttt{if } x < y \texttt{ goto } L_1 \\ & \texttt{goto } L_2 \\ L_1: & \ldots \end{array}$ into $\begin{array}{ll} & \texttt{ifFalse } x < y \texttt{ goto } L_2 \\ L_1: & \ldots \end{array}$

Conclusion

What did we cover today?

- Intermediate Representation generation
- TACs
- SSA
- Control Flow Operators

Next time?

- Runtime environments

Optional material: Further IRs

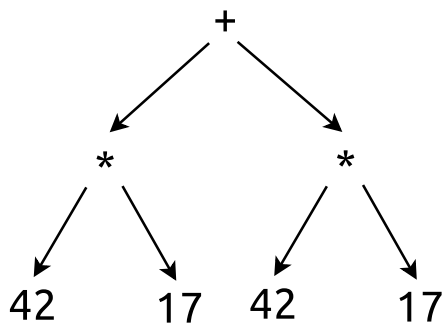
Directed Acyclic Graphs

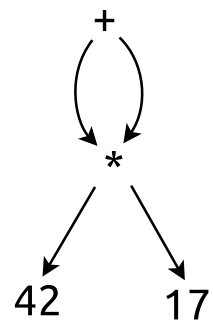
Nodes in a syntax tree represent source program constructs

In an **directed acyclic graph** (DAG), common subexpressions are captured by a single node (spaced optimized trees).

- nodes in a tree have a single parent
- nodes in a DAG can have multiple parents

Example: AST and DAG for $42 + 42 * 17$



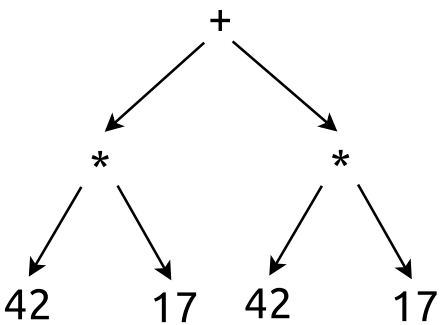


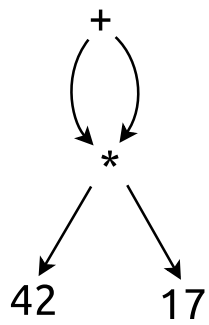
The DAG can be built by a walk through the AST, by keeping track of the nodes that have already been created.

Directed Acyclic Graphs

DAGs can be constructed as follows.

Nodes are stored in an array





1	num	42	
2	num	17	
3	*	1	2
4	+	3	3

- index is called **value number**
- last column stores **the children** or the **lexical value**

The **signature** of a non-leaf node is a triple of the form $\langle op, l, r \rangle$, where op is the node's label, and l and r the value numbers of its 2 children.
For example: $\langle +, 3, 3 \rangle$

How do we know whether to create a new entry or point to an existing one?

Directed Acyclic Graphs

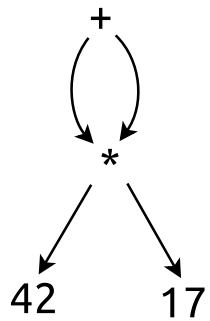
We can search efficiently for matching signatures using a **hash table** that maps **hash values of signatures** to a list of **value numbers**.

For example:

- when we handle the first $*$:
  - we hash $\langle *, 1, 2 \rangle$ to h_1
  - because h_1's bucket is empty, we store $\langle *, 1, 2 \rangle$ in the array
  - we store its value number 3 in h_1's bucket
  - we return the value number 3
- when we handle the second $*$:
  - we hash $\langle *, 1, 2 \rangle$ to h_1 again
  - we lookup h_1 in the hash table and find 3 in its bucket
  - as the array entry 3 matches $\langle *, 1, 2 \rangle$, we return 3 (the array is not extended)
- when we handle $+$:
  - both children return 3
  - we hash the $\langle +, 3, 3 \rangle$ to h_2
  - because h_2's bucket is empty, we store $\langle +, 3, 3 \rangle$ in the array
  - we store its value number 4 in h_2's bucket
  - we return the value number 4

Directed Acyclic Graphs

TACs vs. DAGs/trees: TACs are linear representations of DAGs (or trees) where nodes are explicitly named



$$t_1 = 42 * 17$$
$$t_2 = t_1 + t_1$$

Directed Acyclic Graphs

Build the DAG for $42 + 42 * 17$